Algorítmica

2º Grado en Ingeniería Informática

Guion de prácticas

Algoritmos de exploración de grafos

Índice

1. Objetivo..... 2

2. Problemas..... 2

3. Entrega de la práctica..... 4

Algoritmos de exploración de grafos

1. Objetivo

El objetivo de la práctica consiste en que el alumno sea capaz de analizar un problema, resolverlo mediante la técnica de Backtracking y optimizar la solución utilizando Branch & Bound.

Para ello, los alumnos deberán resolver por equipos los siguientes problemas.

2. Problemas

2.1. Selección de proyectos

Tienes N proyectos, cada uno con un coste y un beneficio, y un presupuesto fijo. Debes elegir una combinación de proyectos que no superen el presupuesto dado.

Por ejemplo: Nos han dado un presupuesto máximo de 8, y tenemos los siguientes proyectos:

Proyecto	Coste	Beneficio
Proyecto A	2	4
Proyecto B	3	5
Proyecto C	5	6

Un ejemplo de combinación sería {A,C} cuyo coste es 7.

Se pide:

1. Implementar un algoritmo de Backtracking que genere todas las combinaciones de proyectos que no superen el presupuesto. Mostrar cada subconjunto válido y su beneficio total, e indicar el número de nodos generados y el número de nodos podados.
2. Implementar Branch & Bound para encontrar la selección de proyectos con máximo beneficio sin pasarse del presupuesto. Se debe de definir claramente:
    a. La representación del estado.
    b. La función de coste.
    c. Una cota superior del beneficio alcanzable desde un nodo
    d. Una cota inferior inicial.
    e. Mostrar la solución óptima, el número de nodos generados y podados, y la estrategia de exploración utilizada.

2.2 Problema del coloreo de grafos

Una empresa de planificación territorial necesita diseñar un mapa de regiones administrativas para su publicación. Cada región debe ser coloreada de forma que no existan dos regiones adyacentes (que compartan frontera) con el mismo color.

El problema se modela como un grafo no dirigido G = (V,E), donde cada vértice representa una región y cada arista indica adyacencia.

Se dispone de:

- Un conjunto de N regiones.
- Una lista de adyacencias (grafo).
- Un conjunto limitado de colores disponibles.

Se pide:

1. Diseñar un algoritmo de backtracking que determine si es posible colorear el mapa usando como máximo K colores, cumpliendo la restricción de que regiones adyacentes no compartan color. El algoritmo debe:
    a. Construir la solución de forma incremental.
    b. Utilizar una función de factibilidad para evitar conflictos.
    c. Mostrar una solución válida (si existe).
    d. Indicar el número de nodos generados y podados.
2. Diseñar un algoritmo de branch and bound que encuentre el mínimo número de colores necesarios para colorear el mapa correctamente. El algoritmo debe:
    a. Definir la representación del estado, la función de coste (número de colores utilizados), una cota inferior del número de colores necesarios desde un estado parcial y una cota superior inicial (por ejemplo, mediante un algoritmo voraz).
    b. Utilizar estas cotas para podar ramas que no puedan mejorar la mejor solución encontrada y así guiar la búsqueda.
    c. Se debe mostrar la solución óptima, el número de nodos generados y podados, y el tamaño máximo de la estructura de nodos vivos.

2.2.1 Material

El material de esta práctica incluye un conjunto de instancias de grafos de prueba y un código base en C++ que ya lee los archivos, construye la representación de grafo y realiza un coloreo voraz. A partir de esa base, los alumnos deben implementar los algoritmos de backtracking y branch and bound para resolver el problema del coloreo con máximo K colores y para encontrar el número mínimo de colores necesarios.

En el código base:

- "Vértices" corresponden a "regiones".
- "Aristas" corresponden a "adyacencias" o "fronteras".
- El grafo se representa con una lista de adyacencias.

Contiene además un fichero de prueba map_coloring_example.txt con el siguiente formato de entrada:

- Primera línea significativa: número de vértices ‘n’ y, opcionalmente, se le pueden pasar el número de aristas ‘m’

- Líneas siguientes: ‘u v’ para cada arista

Y por último, en ejemplo_salida.txt podéis ver cómo sería la salida para el ejemplo de map_coloring_example.txt con 3 colores.

3. Entrega de la práctica

Se deberá entregar un fichero ZIP conteniendo:

- Uno o varios ficheros .cpp por cada problema con el código fuente desarrollado y un único fichero makefile para compilarlos.

- Una memoria detallada en PDF. Debe incluir el nombre de todos los miembros del equipo, así como el porcentaje de participación de cada uno en la práctica.

Para cada problema, la memoria debe incluir:

- Descripción de la solución implementada. Se puede indicar pseudocódigo (no se permite incorporar bloques de código). Muestre la lógica de la solución implementada mediante un caso de uso.

- Análisis de la eficiencia teórica y empírica tanto del algoritmo Backtracking como del algoritmo Branch & Bound, y comparativa entre ellos (nodos generados, nodos podados, tiempo de ejecución…)

- Ejecución del programa implementado. Debe indicar cómo compilar los algoritmos y cómo ejecutarlos para un caso de uso. Aporte gráficas o tablas con distintos casos de uso y solución encontrada mediante el programa.

La práctica deberá ser entregada por PRADO, en la fecha y hora límite indicadas en clase por el profesorado. No se aceptarán, bajo ningún concepto, prácticas entregadas con posterioridad a la fecha límite indicada.